Esercizi con gli operatori logici, indentazione e commenti

Lezione 10 — 1IOT, SST-ICT-01 — Anno bisestile, triangolo valido, allarme con NOT, indentazione, commenti; introduzione al firmware (Arduino, Harvard e Von Neumann)

12/10/2026

1 Da dove ripartiamo

Nota

Dopo la **Verifica 1** della mattina, questa lezione mette in pratica gli **operatori logici** della lezione precedente: **AND** (`&&`), **OR** (`||`) e **NOT** (`!`) dentro la **condizione** dell'`if`. Si scrivono programmi interi in **onlinegdb**, partendo da testi di esercizi della scheda **"V02 - Esercizi Operatori Logici"**. Si introducono poi due **regole di scrittura** (indentazione e commenti) e, negli ultimi minuti, la prima **chiacchierata** sul firmware.

Definizione

Regola di traduzione dalle frasi agli operatori. Leggendo il testo di un esercizio: se trovo **"oppure"** vuol dire **OR**, cioè la doppia barra verticale `||`; se trovo **"e"** vuol dire **AND**, cioè la doppia e commerciale `&&`; se trovo **"non"** vuol dire **NOT**, cioè il punto esclamativo `!`. In inglese "e" si traduce *and*. Il **"diverso"** si scrive con il punto esclamativo seguito dall'uguale, `!=`, e non con = da solo.

2 Esercizio: l'anno bisestile

Esempio

Testo (esercizio 10 della scheda). Scrivere un programma in C che richieda un anno. Se l'anno è divisibile per 400 **OPPURE** (divisibile per 4 **E NON** divisibile per 100), il programma stamperà "Anno bisestile", altrimenti stamperà "Anno non bisestile". *Nota: usa l'operatore* `%` *per la divisibilità, es.* `anno % 4 == 0`.

Passo 1: la variabile per l'anno

Quale **tipo di dato** serve per contenere un anno come 2026? Il docente ragiona ad alta voce con la classe, scartando le alternative:

- un **`char`** (nel corso vale 0–255): **no**, il 2026 non ci sta dentro;
- un **`float`**: **no**, un anno non ha la virgola, non ha senso;
- l'**`int`** è l'**unica scelta sensata**.

Quindi `int anno;`. Poi si chiede il valore all'utente con `printf` ("Inserisci l'anno") e lo si recupera con `scanf` usando `%d` e `&anno`.

Passo 2: la condizione

Il programma deve scegliere se stampare "bisestile" o "non bisestile": serve l'**`if-else`**. Si costruisce la condizione un pezzo alla volta.

1. **Divisibile per 400**: un numero è divisibile per 400 se il **resto della divisione** (operatore **modulo** `%`) tra `anno` e 400 è **zero**, cioè `(anno % 400 == 0)`. Le parentesi si mettono "per sicurezza".
2. Il testo dice "**oppure**": serve l'operatore **OR**, `||`, e dopo di lui un'altra condizione tra parentesi.
3. **Divisibile per 4**: `(anno % 4 == 0)`.
4. Il testo dice "**e**": serve l'operatore **AND**, `&&`.
5. **Non divisibile per 100**: se un numero non è divisibile, il resto della divisione è **diverso da zero**, cioè `(anno % 100 != 0)`.

Esempio

Il codice scritto in classe (con il commento iniziale con il testo dell'esercizio, come copiato da **onlinegdb**):

```c
/*
Anno Bisestile: Scrivere un programma in C che richieda un anno.
Se l'anno e' divisibile per 400 OPPURE (divisibile per 4 E NON divisibile per 100),
il programma stampera "Anno bisestile", altrimenti stampera "Anno non bisestile".
(Nota: usa l'operatore % per la divisibilita', es. anno % 4 == 0)
*/
#include <stdio.h>

int main()
{
    int anno;
    printf("Inserisci l'anno: ");
    scanf("%d", &anno);

    if ( (anno % 400 == 0)  ||  ( (anno % 4 == 0) && (anno % 100 != 0) ) )
    {
        printf("Anno bisestile");
    }
    else
    {
        printf("Anno non bisestile");
    }

    return 0;
}
```

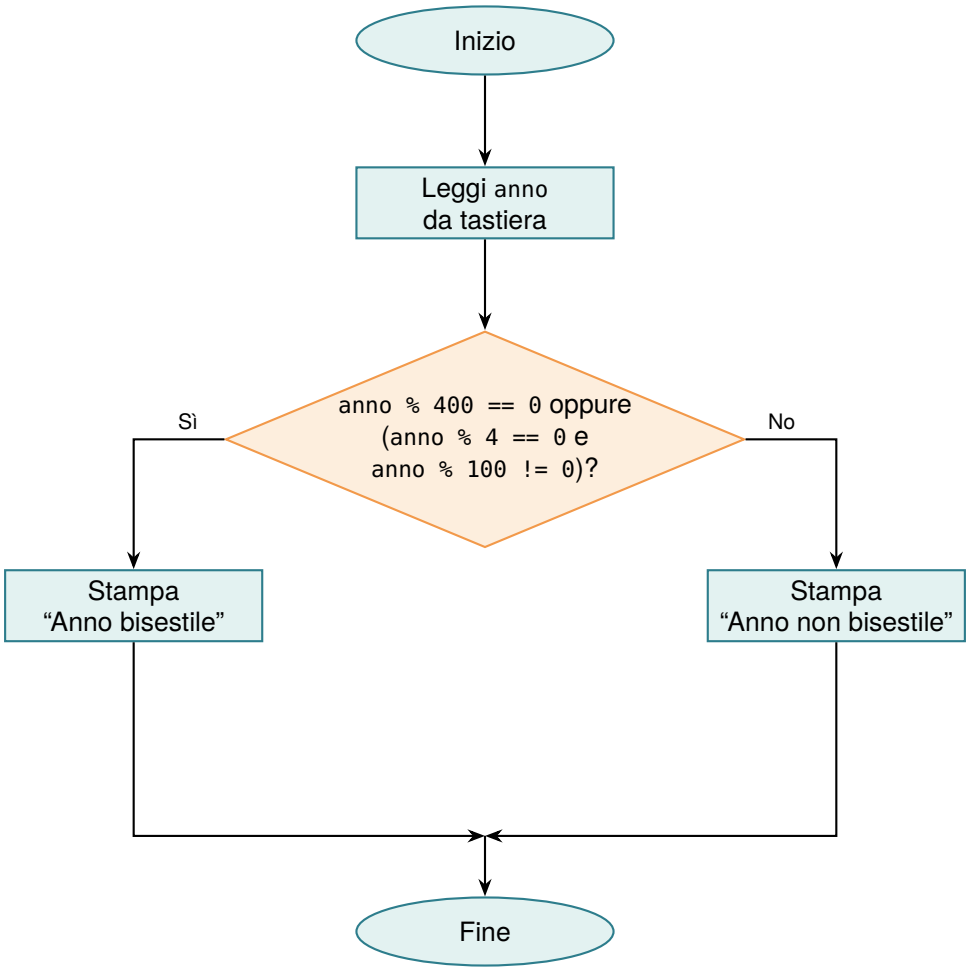


Figura 1: Diagramma di flusso dell'anno bisestile: una sola decisione, con una condizione composta da OR e AND.

Nota

Provare il programma. La classe ha collaudato il programma con vari anni: il **2000** è bisestile (divisibile per 400); il **2025** e il **2026** **non** sono bisestili; anche il 2022 non lo è. Un anno bisestile ha **un giorno in più** (febbraio di 29 giorni). Un errore di battitura del docente (`% 101` invece di `% 100`) è stato segnalato dalla classe e corretto: **controllare sempre i numeri** scritti nelle condizioni.

Attenzione

La regola "**AND** prima, **OR** dopo" non va lasciata al caso: le **parentesi tonde** attorno a ogni **condizione semplice** e attorno al blocco AND rendono la lettura immediata. Vedi la sezione sulla priorità degli operatori più avanti.

3 Una nuova regola: l'indentazione

Definizione

L'**indentazione** è il **rientro** del codice a destra, in modo ordinato. Il docente l'ha scritta alla lavagna ("ho dimenticato una N": **INDENTAZIONE**) e da oggi è **obbligatoria**. Regole:

- la **graffa** che chiude sta **esattamente in verticale** sotto la graffa che apre, nella stessa colonna;
- tutto il codice **contenuto** tra le graffe va scritto una **tabulazione** a destra, per ogni riga: si preme il tasto **TAB** (a sinistra della Q);
- se dentro ci sono altre graffe, il loro contenuto va spostato ancora a destra di una tabulazione.

Il tasto TAB inserisce alcuni spazi (di solito tre, nell'editor usato in aula due o quattro): non si può configurare nell'editor di **onlinegdb**.

Esempio

Lo stesso codice scritto **senza** e **con** indentazione. Nella prima versione non si vede se manca una graffa; nella seconda si vede subito.

```c
if (a < b)
{
printf("minore");
}
else
{
printf("maggiore");
```

```c
if (a < b)
{
    printf("minore");
}
else
{
    printf("maggiore");
```

Con il codice indentato si capisce al volo che **manca la graffa** in fondo a chiudere l'`else`: "apro, apro, chiudo, e devo chiudere anche quell'altra". Con il codice tutto a sinistra l'errore è invisibile.

Attenzione

Perché è obbligatoria. Il codice **funziona lo stesso** senza indentazione (in C si è "fortunati"), ma non è leggibile. In azienda si lavora in gruppo: i **colleghi** devono poter leggere il codice e capire cosa si è fatto. Un file di 15 000 righe senza indentazione è illeggibile e rende la **manutenzione** lenta e piena di problemi. Da oggi qualunque codice consegnato **non indentato vale zero**: "non è che voglio, lo pretendo".

- Errore tipico: due righe di `printf` spostate **troppo a destra** rispetto al contenitore delle

graffe. Funziona, ma è un errore di stile.

- In altri linguaggi come **Python** non ci sono le graffe: il gruppo di istruzioni è definito *solo* dal rientro, e un rientro sbagliato fa **non funzionare** il programma.

4 Esercizio: triangolo valido

Esempio

Testo (esercizio 9 della scheda). Scrivere un programma in C che richieda tre numeri interi relativi ai lati di un triangolo (`a`, `b`, `c`). Se `a < b + c` **E** `b < a + c` **E** `c < a + b`, il programma stamperà "Triangolo valido", altrimenti stamperà "Triangolo non valido".

Definizione

Metodo: il docente ha **ricopiato il testo** dell'esercizio trasformandolo in codice: "se" diventa **`if`**, "e" diventa **AND** (`&&`). Si dichiarano tre **variabili intere**, si chiedono i tre lati con `printf` e `scanf` (i testi sono necessari, "se no non si capisce niente"), poi si scrive la condizione. Dichiarazioni, `printf` e `scanf` riguardano prima `a`, poi `b`, poi `c`.

Esempio

Il codice scritto in classe:

```c
#include <stdio.h>

int main()
{
    int a;
    int b;
    int c;

    printf("Inserisci il primo lato del triangolo: ");
    scanf("%d", &a);
    printf("Inserisci il secondo lato del triangolo: ");
    scanf("%d", &b);
    printf("Inserisci il terzo lato del triangolo: ");
    scanf("%d", &c);

    if ( ( a < b + c)  && (b < a + c)  && (c < a + b) )
    {
        printf("Triangolo valido");
    }
    else
    {
        printf("Triangolo non valido");
    }

    return 0;
}
```

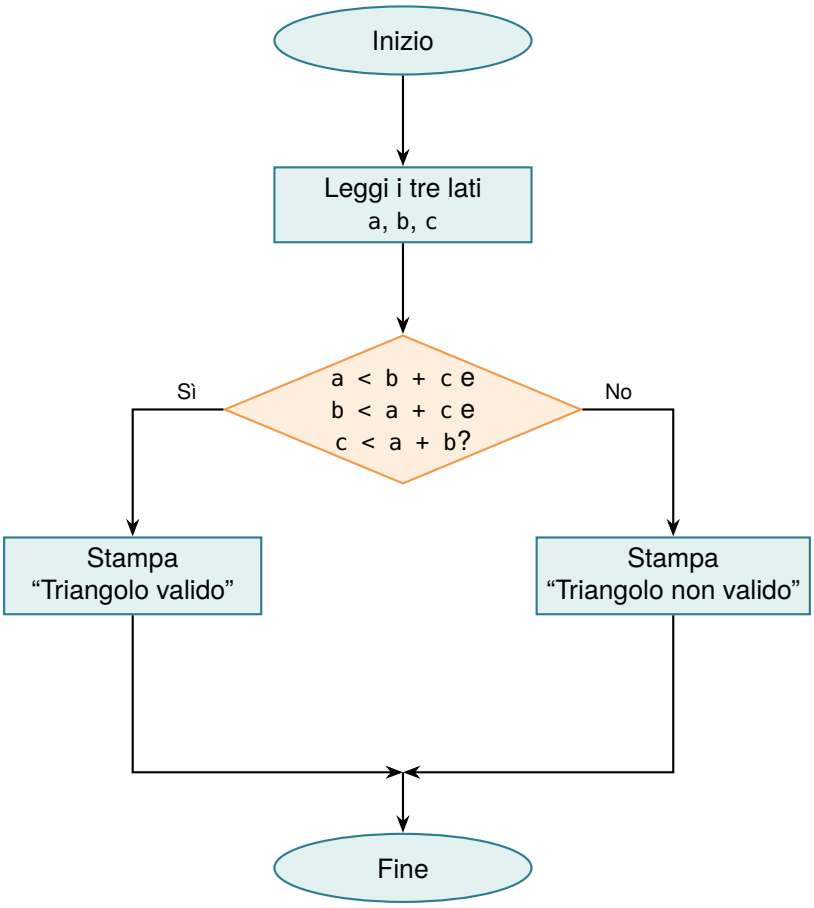


Figura 2: Diagramma di flusso del triangolo valido: tre confronti uniti da due AND.

Nota

Perché questa condizione. In un triangolo ciascun lato deve essere **minore della somma** degli altri due. Il programma verifica le tre disuguaglianze insieme: basta che **una sola** sia falsa perché la **AND** dia **falso** e il triangolo non sia valido. (Se un lato è *uguale* alla somma degli altri due il triangolo “si schiaccia” in un segmento e la condizione stretta lo scarta, correttamente. Il testo dell’esercizio non chiede di controllare che i lati siano positivi.)

Parentesi e priorità degli operatori

Si può scrivere la condizione anche **senza** le parentesi tonde interne (`a < b + c && b < a + c && c < a + b`) e funzionava. Il docente: funziona, ma è **pericoloso**. Il compilatore valuta prima le **somme**, poi i **confronti**, e per ultimo l’operatore logico **AND**. Alcuni potrebbero pensare che cambiando compilatore l’ordine cambi: “hai avuto fortuna”. Il docente le tonde le mette **sempre**, perché così “non c’è possibilità di errore”, e volendo essere ancora più pignoli le metterebbe anche attorno alle somme, `(b + c)`. Così il codice funziona con tutti i compilatori. “Noi lavoriamo su Arduino e in C, oltre non andiamo.”

Attenzione

Per precisione: l’**ordine di valutazione** (**priorità**) degli operatori è fissato dallo **standard del linguaggio C**, non cambia da compilatore a compilatore: prima gli operatori aritmetici, poi i

confronti, poi `&&`, poi `||`. La regola pratica del docente resta valida: le **parentesi** tolgono ogni dubbio a chi legge il codice (e a chi lo scrive).

5 Esercizio: allarme con l'operatore NOT

Esempio

Testo (scheda di esercizi, copiato nell'editor). *Allarme Disattivato*: scrivere un programma in C che richieda all'utente lo stato dell'allarme (1 per attivo, 0 per disattivato). Usando l'operatore NOT (`!`), se l'allarme NON è attivo il programma stamperà "Casa sicura"; altrimenti "Allarme in corso" (il testo nell'editor si legge solo in parte).

La variabile e l'input

Per memorizzare 1 oppure 0 basta un **`char`**: non serve un `int`. Il docente scrive `char allarme;`, poi un `printf` ("Inserisci lo stato dell'allarme (1 attivo, 0 disattivato)") e un `scanf` con `%d` e `&allarme`. Si può usare anche un `int`: "va bene, non cambia niente", perché quello che interessa subito è valutare se l'allarme è acceso o spento, **vero o falso**. (Un caso più avanzato è il keypad, il tastierino: richiede di leggere i pulsanti, argomento che il docente spera di raggiungere verso la fine dell'anno.)

Attenzione

Nota tecnica: `%d` legge un **intero** (di solito 4 byte) e lo scrive nella variabile; se la variabile è un **`char`** (1 byte) la scrittura va **oltre** la variabile e il comportamento non è definito dallo standard. Per un `char` il formato corretto è `%hhd`. Per questo, nel codice sotto, la variabile è un **`int`**, scelta ammessa dal docente ("non cambia niente").

La condizione con NOT

Se l'allarme *non* è attivo, dentro la variabile c'è **zero**. Con l'operatore NOT: se `allarme` vale 0, `!allarme` vale **1**, cioè vero. Il docente scrive la condizione come `!allarme != 0`, cioè "il NOT dell'allarme è **diverso da zero**": risolve la questione. Avrebbe potuto scrivere anche `== 1`, ma "quando uso il NOT lo fa semplicemente diventare vero: da nessuna parte c'è scritto che lo fa diventare uno, o almeno non sempre". Si poteva usare anche il **maggiore di zero**, ma il **diverso da zero** è preferibile: "è più facile, perché il numero può essere positivo o negativo, quello che vuoi".

Esempio

Programma completo (ricostruito dalla spiegazione; il testo scritto in aula non è visibile per intero nei frame):

```c
#include <stdio.h>

int main()
{
    int allarme;
```

```
    printf("Inserisci lo stato dell'allarme (1 attivo, 0 disattivato): ");
    scanf("%d", &allarme);

    if ( (!allarme) != 0 )
    {
        printf("Casa sicura");
    }
    else
    {
        printf("Allarme in corso");
    }

    return 0;
}
```

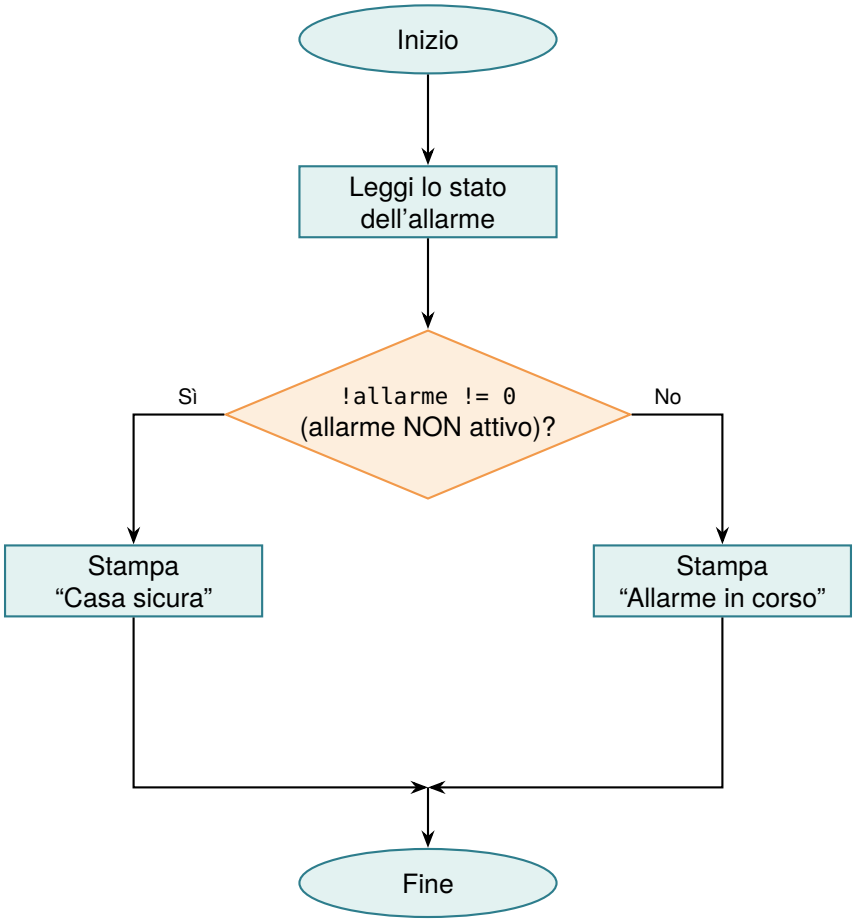


Figura 3: Diagramma di flusso dell'allarme: la condizione usa l'operatore NOT.

La scrittura da firmware: `!allarme` senza confronto

La condizione si può scrivere anche **senza** il confronto con zero, lasciando solo `!allarme`, e funziona. Perché? Non è perché "inverte comunque": è una tecnica che usano tantissimi **firmwaristi**, "io per primo scrivo così", cioè **scrivere meno**. Con la classe il docente ha scritto il **diverso da zero** perché prima servono le basi, non le tecniche più avanzate.

Definizione

La **condizione** dell'`if` deve dare come risposta **vero o falso**. Gli operatori condizionali (`a > b`, `x != 0`, ...) producono vero o falso; ma anche `!allarme` dà già da solo vero o falso. Quindi **non serve** il confronto con zero: `if (!allarme)` basta. La regola che sta sotto è quella già vista: **zero = falso**, **qualunque altro numero = vero**.

Attenzione

Esempio del rischio: se nella variabile c'è 42, 42 è **vero** e negato diventa **falso**: il programma esegue sempre il ramo `else`. Non si dice "positivo" o "negativo": si dice **vero** o **falso**. La scrittura corta funziona benissimo, ma "è una cosa un pochino più evoluta: il rischio di interpretarla male è molto alto". A meno di essere ben sicuri, **attenzione**: "l'errore è dietro l'angolo".

Graffe facoltative con una sola istruzione

Se dopo l'`if` o l'`else` c'è **una sola istruzione** le **graffe** si possono omettere e il programma funziona lo stesso. Noi le **mettiamo comunque**, perché il codice è più leggibile: "evitate queste cose, se possibile".

Nota

Valutazione degli esercizi. Come verranno valutati gli esercizi di programmazione? una verifica che chiede "rispondere a queste domande" ha pochi criteri; la **prossima verifica** sarà "**scrivi un programma** che faccia questo, questo e quest'altro". Un programma o funziona o non funziona, ma in C può anche funzionare *per caso* (si è visto con le parentesi). Il docente vuole dare spazio alla capacità di avere scritto il programma, ma conta anche l'indentazione e la qualità. Se la verifica è **sul computer**, consegnare un programma che **non compila** è pesante: il compilatore mostra l'errore e non saperlo correggere significa non aver capito come funziona; su carta una virgola dimenticata pesa meno. "Il voto è l'obbligo istituzionale che abbiamo."

6 I commenti nel codice

Definizione

Un **commento** è un testo scritto nel codice che il **compilatore non considera**. Serve a chi legge il codice per capirci qualcosa. In C si può scrivere in **due modi**:

- **commento multilinea**: tra `/*` e `*/`. Tutto quello che sta tra i due simboli è commento, anche su più righe;
- **commento su una riga**: dopo `//`. Vale solo fino a fine riga: quando si va a capo ricomincia il **codice normale**.

Esempio

Alla lavagna il docente ha disegnato i simboli: `/*` (apertura), `*/` (chiusura) e `//` con una riga a fianco. Nell'editor di **onlinegdb** tutto il testo commentato cambia **colore** (arancione): il **commento multilinea** del programma (come nell'intestazione di `main.c`) parte dalla riga 1 e termina alla riga 8 con `*/`.

```
/* Questo e' un commento
   su piu' righe: tutto quello che
   scrivo qui il compilatore lo ignora */

// Stampo un messaggio
printf("Ciao");

// Chiedo all'utente un numero
scanf("%d", &x);
```

Il docente ha mostrato come si possono mettere commenti prima di un `printf` ("stampo un messaggio") o prima di una `scanf` ("chiedo all'utente..."): **scrivete quello che volete**, a fine riga si torna a scrivere codice. Se invece si apre con `/*`, tutto diventa commento **finché non si chiude** con `*/`.

Attenzione

I commenti in azienda sono obbligatori. Chi scrive un software e non lo commenta se lo vede **rimandare indietro** dal titolare ("vedi, ci metti i commenti, altrimenti non vado avanti"): domani chi l'ha scritto potrebbe non lavorare più lì e il codice è **dell'azienda**, che deve continuare a venderlo e a mantenerlo. Chi arriva dopo, senza i commenti, non sa cosa c'è scritto.

7 Cambiamo argomento: che cos'è il firmware

Nota

Il docente apre una **"chiacchierata di 20 minuti"** sulla seconda materia del corso: il **firmware**. Finora, nella materia di programmazione, si è lavorato **al computer**. Fare firmware vuol dire **scrivere programmi** che non girano sul computer, ma fanno funzionare delle **schede elettroniche**, come quella mostrata il secondo giorno di scuola. In primo anno si usa **Arduino**; il docente conta di abbandonarlo entro la fine dell'anno per qualcosa di più interessante.

Esempio

Perché Arduino è solo didattico. Il docente non ama Arduino perché "semplifica troppo le cose": è così facile che non serve alcuna competenza, come un frullatore di casa. Va bene per **sperimentare** e capire concetti che altrimenti sarebbero molto complessi, ma **non è fare firmware**. La scheda nasce **ufficialmente nel 2005** (con sistemi prototipo prima): la si collega al computer con un **cavo USB**, le si scrive cosa fare e lo fa.
Prima, il docente lavorava con i soli chippettini neri, il **microcontrollore** (il "cervello" della scheda), senza scheda: serviva comprare il **manuale** (circa 70–80 mila lire, 80 euro di oggi),

studiarlo, scrivere circa 200 righe di codice *solo* per impostare la macchina e poi concentrarsi su cosa farle fare. Difficoltà d'ingresso altissima: Internet poco, YouTube non esisteva, nessuno spiegava le cose. Con questa scheda tutto è diventato semplice, ma provate ad aprire il **frigorifero**, il **forno**, il **microonde**, la TV, il cellulare: dietro non c'è una scheda del genere. Va bene a livello **didattico**, non a livello di **produzione**. Chi imparasse solo Arduino in quattro anni "non lo assume nessuno".

Definizione

Il **microcontrollore** è un **chip nero** con tanti **piedini** di metallo. Ogni piedino è un **contatto elettrico**, cioè un filo che la macchina può accendere o spegnere: se do corrente è **1**, se tolgo la corrente è **0**. In base ai livelli accesi/spenti sui piedini succede qualcosa **all'esterno**. Dentro il chip si carica il **programma** scritto da noi e i **dati** da elaborare.

Definizione

CPU = *Central Processing Unit*, **unità centrale di elaborazione**: "il cervello", quello che fa i **calcoli** e sceglie quale istruzione eseguire. Nel chip è collegata a **due memorie**:

- la **Program Memory** (memoria programma): contiene il **codice** scritto dal programmatore;
- la **Data Memory** (memoria dati): contiene i **dati**, cioè tutte le **variabili**. Ogni volta che si crea una variabile (`int x;`, `int allarme;`) questa finisce nella memoria dati.

In **onlinegdb** si scrive il codice e si pensa che finisca lì, ma la variabile vera e propria viene creata nella memoria dati del dispositivo.

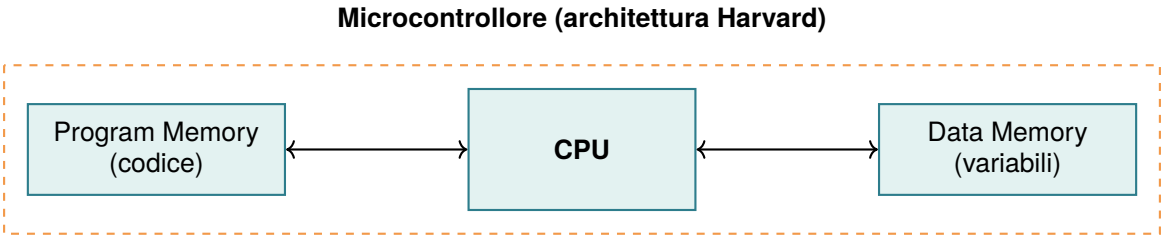


Figura 4: Schema a blocchi di un microcontrollore: una CPU, una memoria per il programma e una memoria per i dati, separate.

Nota

Domanda dalla classe, non svolta: esiste una tecnologia che sposta i dati in memoria senza occupare la CPU. Il docente: "troppo complesso da fare in questo momento", roba molto **specialistica** (non è argomento del primo anno); eventualmente se ne parla a ricreazione o in un altro momento.

8 Quanta memoria? Confronto con il PC

Esempio

PC di casa. Sul computer degli allievi: **RAM** di 32 e 128 gigabyte (128 gigabyte = 128 con nove zeri di byte); **disco** da 500 gigabyte fino a 4, 6 e 12 terabyte. Sull'**Arduino** la memoria dati ha solo **2 kilobyte**. Il docente ha controllato il **datasheet** (la scheda tecnica) del microcontrollore dell'Arduino Uno, l'**ATmega328P** dell'Atmel (ora Microchip), dal sito del costruttore: **32 KB** di memoria **flash** per il programma, **1 KB** di **EEPROM**, **2 KB** di **SRAM** interna per i dati. "Sono tanti o pochi?" Un computer ha una memoria **infinitamente più grande**: "rapporti di lavoro completamente diversi".

	Arduino (ATmega328P)	**PC di casa (esempi in aula)**
Memoria dati (RAM)	2 KB (SRAM)	32–128 GB
Memoria programma / disco	32 KB (flash)	4–12 TB (disco)

9 Architettura Harvard e architettura Von Neumann

Definizione

Un dispositivo con **due memorie separate**, una per il **programma** e una per i **dati**, ha un'**architettura Harvard**. È una scelta di progetto ("dobbiamo costruire un nuovo tipo di computer: facciamolo così").

Definizione

Un dispositivo con la CPU e **una sola memoria** che contiene sia i **dati** sia i **programmi** ha un'**architettura Von Neumann**. Esempio: i **PC**. Un programma salvato sul disco, per funzionare, viene caricato in **RAM**: la memoria con cui scambia dati la CPU è solo la RAM.

Esempio

Quando si lavora al **PC** (onlinegdb) c'è l'architettura **Von Neumann**; quando si lavora solo con l'**Arduino** c'è l'architettura **Harvard**. **Domanda da verifica**: "Qual è la differenza tra una macchina di tipo Harvard e una di tipo Von Neumann?" Risposta: **Harvard** ha **due memorie**, una per i dati e una per il programma; **Von Neumann** ha **una sola memoria** dove sta tutto.

Nota

Qual è la migliore? L'**Harvard** è più **veloce**: la CPU può scambiare dati con la memoria programma e con la memoria dati **contemporaneamente** (carica il programma mentre lavora sui dati). Nella Von Neumann, invece, o si legge il **programma** o si leggono i **dati**: si lavora, in un certo senso, "a metà velocità".

10 Perché i PC non sono Harvard: le scelte del mercato

Esempio

Perché allora i PC non sono costruiti con la Harvard? un computer Harvard costerebbe **molto di più**; "tu poi non lo compri e se tu non lo compri io non lo produco". Le **scelte tecnologiche** non sono mai le migliori: sono quelle che **vendono**: se un prodotto costa troppo non ha senso farlo. Se un computer "entry level" costasse 7000 euro si perderebbe tutta la **fetta di mercato**.

Esempio

Il caso dei processori (esempio delle fasce di velocità). Un Intel i7, a parità di generazione e di numero di **core**, esiste in versioni a 3, a 4 e a 5 GHz. Intel ha una linea di produzione per ogni velocità? **No**: la linea è una sola e **produce** processori pensati per i **5 GHz**. All'uscita dalla fabbrica ogni chip viene **testato**:

- se arriva a 5 GHz, si stampa 5 GHz e lo si vende a prezzo più alto;
- se si ferma a 4 GHz ("è difettoso e non riesce ad andare a 5"), si stampa 4 GHz;
- se si ferma a 3 GHz, si stampa 3 GHz e si vende a prezzo più basso.

Spreco zero: tutto quello che si produce si vende. Chi compra la versione più lenta compra in senso tecnico un prodotto "difettoso" di cui il produttore è consapevole: a 3 GHz non si vede, ma se si prova l'**overclock** la macchina "si impalla prima". Questo vale per la *stessa* famiglia: Ryzen 7 e Ryzen 9 sono invece **due linee diverse** (processi un po' diversi). Per altre sigle (come i modelli "KF") il docente dice di non saperne parlare: è un mondo che non lo interessa. Qui interessa che la **linea di produzione** è una sola e la differenza di velocità è una **selezione** in base a come è venuto il chip ("venuto bene? lo vendiamo a più soldi").

11 Compiti e consegne

Attenzione

Relazione su Von Neumann: il docente si aspetta per **venerdì** o per **lunedì prossimo** una **breve relazione** su **cosa ha fatto Von Neumann** ("non so, se vi interessa lo cercate"). Il docente avrebbe voluto mostrare un film, ma non ce l'ha. Altri compiti: "domani mattina non avete niente al momento" e "tutto il codice lo troverete nella dispensa di stasera": gli esercizi di questa lezione (anno bisestile, triangolo, allarme) sono quelli della scheda *V02 — Esercizi Operatori Logici*, che prosegue con le altre voci (5 temperatura di allarme, 6 giostra al Luna Park, 7 numero pari e positivo, 8 accesso con password e PIN).

Esempio

Testi esatti delle voci 5–8 della scheda (visibili a schermo, non svolti in aula).

- **5. Temperatura di Allarme.** Scrivere un programma in C che richieda all'utente di inserire la temperatura della stanza. Se la temperatura è minore di 18 **OPPURE** maggiore di 26, il programma stamperà "Allarme temperatura", altrimenti stamperà "Temperatura OK".

- **6. Giostra al Luna Park.** Scrivere un programma in C che richieda all'utente l'altezza in cm e l'età. Se l'altezza è maggiore o uguale a 140 **E** l'età è maggiore o uguale a 10, il programma stamperà "Puoi salire", altrimenti stamperà "Non puoi salire".
- **7. Numero Pari e Positivo.** Scrivere un programma in C che richieda all'utente di inserire un numero intero. Se il numero è maggiore di 0 **E** il resto della divisione per 2 è uguale a 0 (`numero % 2 == 0`), il programma stamperà "Positivo e pari", altrimenti stamperà "Condizione non verificata".
- **8. Accesso con Password e PIN.** Scrivere un programma in C che richieda all'utente di inserire un codice di sicurezza e un PIN (due numeri interi). Solo se il codice è uguale a 1234 **E** il PIN è uguale a 5678, il programma stamperà "Accesso consentito", altrimenti stamperà "Accesso negato".

12 Cosa ricordare di questa lezione

- **"Oppure"** = **OR** (`||`); **"e"** = **AND** (`&&`); **"non"** = **NOT** (`!`); **"diverso"** = `!=`.
- **Anno bisestile**: divisibile per 400, **oppure** divisibile per 4 **e non** per 100; la divisibilità si controlla con il **modulo** `%` uguale a zero.
- **Triangolo valido**: ogni lato è **minore della somma** degli altri due, tre confronti uniti da AND.
- **Zero è falso**, ogni altro numero è **vero**; `!allarme` è vero solo se `allarme` vale **zero**.
- **Indentazione** obbligatoria: graffa che chiude sotto quella che apre, contenuto **una tabulazione a destra**; codice non indentato **vale zero**.
- **Parentesi tonde** sempre nelle condizioni composte; **graffe** sempre, anche con una sola istruzione.
- **Commenti**: `/* ...*/` multilinea, `//` su una riga; il **compilatore** li ignora; in azienda sono **obbligatori**.
- **Firmware**: programmi che fanno funzionare **schede elettroniche**; il **microcontrollore** ha una **CPU** e due memorie (programma e dati).
- **Harvard**: due memorie separate, più veloce (Arduino). **Von Neumann**: una sola memoria (PC).